Analytics for Unstructured Data (F2026)
Assignment 2

Building an Intelligent Crowdsourced Recommender: From Keywords to Natural-Language Intent

New due date: 6th October, by 11:59 p.m.

High-level description

The objective of this group assignment is to create the building blocks of a crowdsourced recommender system for either games or movies. Rather than asking a user to choose a fixed set of attributes, your system should accept a natural-language description of what the user is looking for and recommend three products that best match that request.

The central learning objective is to compare progressively richer ways of understanding user intent: (1) bag-of-words (BoW), (2) pretrained word vectors, (3) custom word vectors learned from your review corpus, and (4) an LLM used as an intent interpreter together with your custom-vector recommendation engine. The LLM should not simply make the recommendations for you.

Example user request

"I'm looking for a game with a really immersive world and a strong story. I enjoy exploration and challenging combat, but I don't want something that feels repetitive or grindy."

For movies, an analogous request might be: *"I want an emotionally engaging movie with strong acting and beautiful cinematography. I like suspense, but I don't want something excessively violent or very slow."*

Task A. Build the review corpus

Scrape approximately 7,500 reviews and ratings covering about 250 games from Steam.com OR movies from RottenTomatoes.com. Your final dataset should retain enough information to identify the product, review text, and rating (where available). Briefly describe your scraping approach, cleaning decisions, and the final number of usable reviews and products.

Task B. Explore the language of the reviews

Before building the recommender, learn what users actually talk about. Conduct a word-frequency analysis to identify important product attributes in your corpus. Examples for games might include story, combat, graphics, exploration, difficulty, multiplayer, progression, replayability, performance, and controls. Examples for movies might include acting, plot, characters, pacing, cinematography, humor, suspense, romance, soundtrack, and ending.

Then use lift analysis on selected attributes to examine which concepts tend to be co-mentioned. This can help you understand whether seemingly reasonable user preferences are actually represented together in

your review data. Explain what you learn from the frequency and lift analyses and how that informs the recommender.

Task C. Recommender 1 — Bag-of-Words + cosine similarity

Use a natural-language user request as the query. Build a BoW representation of the query and the review text, and use cosine similarity to identify products whose review language most closely matches the request. For Task C, treat the user's full natural-language request as a document: apply ordinary text preprocessing and build the BoW vector directly from the processed request. Do not first use NLP or an LLM to extract a separate list of preferred attributes; structured intent interpretation is introduced later in Task F.

Incorporate appropriate sentiment analysis so that similarity alone does not drive the recommendation. For example, a review may use language that closely matches the user's request but express a negative opinion about the product. In Task C, because you are not extracting a separate list of preference attributes, sentiment does not need to be computed separately for each preferred attribute. Instead, use VADER, an LLM, or a combination to estimate sentiment for the relevant reviews, and combine similarity and sentiment in a reasonable way to produce the final recommendation score. Explain your approach clearly in plain English.

Recommend three products. Explain your method in detail in words, including preprocessing, aggregation, similarity, sentiment, and how you combine these elements into the final ranking. Show a table containing your three final recommendations plus at least 20 other top contenders, with enough intermediate scores to make the ranking understandable.

Task D. Recommender 2 — Pretrained word vectors

Repeat the recommendation task using pretrained word vectors (for example, spaCy medium-sized pretrained vectors) rather than plain BoW. Explain how you represent both the natural-language query and the reviews or products.

Compare the recommendations with the BoW results. Do semantic relationships help? For example, can "challenging combat" match reviews discussing "difficult fights" even when the exact words differ? Conversely, do pretrained vectors sometimes introduce broad semantic similarity that is not useful for this particular recommendation problem?

Which method gives better recommendations in your context—BoW or pretrained vectors? Do not automatically assume that word embeddings must be better. Justify your conclusion using your results.

Task E. Recommender 3 — Custom word embeddings

Create custom word embeddings from your own product-review corpus using an appropriate package such as Gensim. Use these domain-specific vectors to repeat the recommendation task. Explain your modeling choices.

Before looking only at the final recommendations, inspect several meaningful word neighborhoods. For a game corpus, for example, does the model learn relationships such as co-op–multiplayer, lag–performance, or grind–progression? For movies, does it learn useful domain relationships around acting, pacing, cinematography, suspense, or character development?

Do your top-three recommendations change relative to BoW and pretrained embeddings? Explain why custom embeddings help, hurt, or make little difference. Remember that a corpus of approximately 7,500 reviews is relatively small for learning embeddings, so limitations should be discussed.

Task F. Recommender 4 — LLM intent interpretation + custom vectors

Now add an LLM as an intent-interpretation layer. The LLM should translate the user's natural-language request into a structured representation of preferences. It should NOT simply be asked to recommend three games or movies.

Concept	Preference	Importance
Immersion	positive	High
Exploration	positive	High
Story	positive	High
Challenging combat	positive	Low
Repetitive / grindy	negative	Medium

The table above illustrates one possible interpretation of the sample game request. You may design a different structured representation, but explain it clearly. Once the LLM has identified the attributes or concepts that matter to the user, your sentiment analysis should also become attribute-specific where appropriate. In other words, do not rely only on overall product sentiment: assess whether reviewers are positive or negative about the particular attributes the user values (for example, story, exploration, combat, or repetitiveness), and take the user's preference direction into account. Then use your custom-vector model, together with this attribute-level sentiment and preference direction, to retrieve and rank products. Explain your approach to sentiment analysis in this context.

Compare this fourth system with the first three. Where does the LLM add value? Pay particular attention to paraphrases, implicit preferences, negation, and statements such as "I don't care about multiplayer." Also identify cases where the LLM interpretation may be questionable or unnecessarily complicated.

Task G. Popularity versus personalization

Ignore query similarity and feature-level sentiment and simply choose the three highest-rated products in your entire dataset. How do these recommendations differ from your personalized recommendations? Would the highest-rated products actually satisfy this user's request? Why or why not? Justify your answer with analysis.

This comparison should help distinguish "What does the crowd like?" from "What appears to fit this particular user?"

Task H. Product similarity and competition

Choose any 10 products from your dataset. Select one focal product and identify the most similar product among the remaining nine. Explain your method, the representation you use, and why the resulting product can reasonably be viewed as similar or competitive.

An interesting result may be a product that is not an obvious competitor based on genre or category but is discussed by customers in similar ways. This is particularly relevant in long-tail markets.

Suggested reading: Josh Barua, "Who is Your Competitor in the Era of the Long Tail?" DataDrivenInvestor
Who is Your Competitor in the Era of the Long Tail? | by Josh Barua | DataDrivenInvestor

What your final comparison should address

Your report/notebook should not merely show four sets of recommendations. Use the comparison to explain what each successive representation contributes. In particular, discuss lexical matching (BoW), general semantic matching (pretrained vectors), domain-specific semantic matching (custom vectors), and intent-aware semantic matching (LLM interpretation + custom vectors).

There is no requirement that the most sophisticated method must win. A strong analysis may find that a simpler method works better for some queries or attributes. The quality of your reasoning, diagnostics, and evidence is more important than producing a predetermined result.

Helpful suggestions

- Use the same natural-language user request across the four recommendation methods so that the comparison is meaningful.
- Choose a request that contains several positive preferences and at least one negative or avoidance preference.
- Do not let the LLM bypass the recommender by directly naming products. Its role in Task F is to interpret intent.
- Keep intermediate scores. Your instructor should be able to understand why product #3 outranked product #4.
- Consider whether sentiment should be measured overall or around specific attributes/concepts; explain your choice.
- Inspect failure cases. A surprising or poor recommendation can be more informative than a successful one if you explain why it occurred.
- Be explicit about how multiple reviews for the same product are combined into a product-level recommendation score.
- For custom embeddings, report relevant training choices and recognize the limitations imposed by corpus size.

Submission

Submit one Python notebook per group containing your code, outputs, tables, answers, and written justifications. The notebook should be readable as an analysis, not merely a sequence of code cells. Clearly label each task and make your recommendation logic reproducible.